Formal Verification Report

Aave V4 Libraries

March 2026

Prepared for Aave Labs

Table of contents

Project Summary

Project Scope

Project Name	Repository (link)	Platform
Aave V4	https://github.com/aave/aave-v4	EVM

Project Overview

This document describes the specification and verification of **Aave V4 Libraries** using the Certora Prover. The document is part of three documents describing the work undertaken from **July 28, 2025** to **March 09, 2026.**

This is one of three formal verification reports, focusing on the **Libraries** component.
The following contract list is included in this scope:

```
src/dependencies/solady/LibBit.sol
src/spoke/libraries/PositionStatusMap.sol
src/spoke/libraries/LiquidationLogic.sol
src/spoke/libraries/SpokeUtils.sol
src/hub/libraries/Premium.sol
src/hub/libraries/SharesMath.sol
src/libraries/math/WadRayMath.sol
src/libraries/math/PercentageMath.sol
src/libraries/math/MathUtils.sol
```

The properties proven in this scope serve the modularity of the proof effort: by establishing each library's correctness in isolation, they allow the companion Hub Formal Verification Report and Spoke Formal Verification Report to rely on symbolic abstractions of these libraries instead of re-verifying their internals.

The Certora Prover demonstrated that the implementation of the above Solidity libraries are correct with respect to the formal rules written by the Certora team.

Assessment Methodology

Formal Verification

We performed verification of the AAVE V4 protocol using the Certora verification tool which is based on Satisfiability Modulo Theories (SMT). In short, the Certora verification tool works by compiling formal specifications written in the Certora Verification Language (CVL) and AAVE's implementation source code written in Solidity.
More information about Certora's tooling can be found in the Certora Technology Whitepaper.

If a property is verified with this methodology it means the specification in CVL holds for all possible inputs. However specifications must introduce assumptions to rule out situations which are impossible in realistic scenarios (e.g. to specify the valid range for an input parameter). Additionally, SMT-based verification is notoriously computationally difficult. As a result, we introduce overapproximations (replacing real computations with broader ranges of values) and underapproximations (replacing real computations with fewer values) to make verification feasible.

Rules. A rule is a verification task possibly containing assumptions, calls to the relevant functionality that is symbolically executed and assertions that are verified on any resulting states from the computation.

Inductive Invariants. Inductive invariants are proved by induction on the structure of a smart contract. We use constructors as a base case, and consider all other (relevant) externally callable functions that can change the storage as step cases.
Specifically, to prove the base case, we show that a property holds in any resulting state after a symbolic call to the respective constructor. For proving step cases, we generally assume a state where the invariant holds (induction hypothesis), symbolically execute the functionality under investigation, and prove that after this computation any resulting state satisfies the invariant.

Security Context

The properties verified in this report target the subset of identified threats that can be addressed at the library level:

- accounting and share consistency (ShareMath, P-06),
- arithmetic correctness (WadRayMath, PercentageMath, MathUtils, P-07),

- liquidation calculation soundness (LiquidationLogic, P-02), and
- the integrity of supporting primitives such as bit operations (LibBit, P-01), position status tracking (PositionStatusMap, P-03), premium computation (Premium, P-04), and value conversion (SpokeUtils, P-05).

Interest accrual correctness is indirectly covered, as the accrual functions depend on the math library primitives proven here. System-level concerns such as hub-spoke isolation, fund transfer security, admin compromise, oracle feed quality, and economic attack surfaces are outside the scope of this report and are addressed in the companion reports.

Collaboration and Fix Validation

Collaboration with the development team runs throughout the engagement to confirm expected behaviors, clarify design assumptions, and ensure accurate understanding of the protocol's intended operation. All findings are documented with clear reasoning, reproducible examples, and actionable recommendations. A follow-up review validates applied fixes and verifies that no regressions or secondary issues have been introduced.

certora

aave

Findings Summary

The table below summarizes the findings of the review, including type and severity details.

Severity	Discovered	Confirmed	Fixed
Critical	-	-	-
High	-	-	-
Medium	-	-	-
Low	-	-	-
Informational	-	-	-
Total	-	-	-

Severity Matrix

Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Low	Low	Medium
		Low	Medium	High
		Likelihood		

Formal Verification

Verification Notations

Formally Verified	The rule is verified for every state of the contract(s), under the assumptions of the scope/requirements in the rule.
Formally Verified After Fix	The rule was violated due to an issue in the code and was successfully verified after fixing the issue

General Assumptions and Simplifications

- We use our own implementation of `Math.mulDiv`, `Math.mulDivUp` and `Math.mulDivDown`. These are equivalent to the originals and better suited for verification purposes.
- Loops are inherently difficult for formal verification. We handle loops by unrolling them a specific amount of times. We use a **loop_iter of 3**, unrolling each loop up to three times.
- Underlying assets are assumed to be standard ERC20, specifically:
    a. No fees on transfer
    b. Without double entries
    c. Decimals between 6 and 18
- The price of assets is assumed to be non-zero.
- The verified Solidity contracts are compiled with solc8.28 (with via-ir).

Properties Overview

ID	Contract	Title	Impact
P-01	LibBit	LibBit functions integrity	broken model, integrity of verification
P-02	LiquidationLogic	Function calculateLiquidationAmounts() and calculateLiquidationBonus()	Direct fund loss, potential insolvency
P-03	PositionStatusMap	PositionsStatusMap integrity	broken model, integrity of verification
P-04	Premium	Correctness of calculatePremiumRay summary	broken model, integrity of verification
P-05	SpokeUtils	Correctness of toValue summary	broken model, integrity of verification
P-06	ShareMath	Integrity of ShareMath functions	broken model, integrity of verification
P-07	Math libraries	Integrity of Math Functions	broken model, integrity of verification

Detailed Properties

LibBit Library

P-01. LibBit functions integrity	
Status: Formally Verified	

Rule Name	Status	Description	Link to rule report
popCount_integrity	Formally Verified	Verifies that popCount(x) correctly counts the number of set bits by flipping bits and checking the delta.	*Run link*
popCount_noRevert	Formally Verified	Ensures that popCount never reverts for any input.	
fls_integrity	Formally Verified	Verifies that fls(x) correctly identifies the position of the most significant set bit	
fls_noRevert	Formally Verified	Ensures that fls never reverts for any input.	
isBitTrueNeverRevert	Formally Verified	Ensures that isBitTrue never reverts for any position	

LiquidationLogic Library

P-02. Function calculateLiquidationAmounts() and calculateLiquidationBonus()

Status: Formally Verified After Fix

Rule Name	Status	Description	Link to rule report
debtToLiquidateNotExceedBalance	Formally Verified	Debt to liquidate cannot exceed user's current debt shares	*Run link*
debtToLiquidateNotExceedDebtToCover	Formally Verified	Debt to liquidate (in assets) cannot exceed debt to cover	
collateralToLiquidatorNotExceedTotal	Formally Verified	Collateral to liquidator cannot exceed collateral to liquidate	
collateralToLiquidateValueLessThanDebtToLiquidate	Formally Verified After Fix	Collateral to liquidate value less than debt to liquidate value. Checked on specific decimals for debt and collateral tokens	*Run Link* *18-18* *Run link* *16-12*
drawnSharesZeroed_premiumDebtRayZeroed	Formally Verified	Drawn shares zeroed implies premium debt ray zeroed	*Run link*
maxBonusWhenLowHealthFactor	Formally Verified	When healthFactor <= healthFactorForMaxBonus, returns maxLiquidationBonus	*Run link*
bonusIsAtLeastNoBonus	Formally Verified	Result is always >= PERCENTAGE_FACTOR (no negative bonus)	
bonusDoesNotExceedMax	Formally Verified	Result is always <= maxLiquidationBonus	
monotonicityOfBonus	Formally Verified	Monotonicity: higher healthFactor results in lower or equal bonus	
bonusAtThreshold	Formally Verified	At threshold, bonus equals minLiquidationBonus	
zeroBonusFactorMeansNoMinBonus	Formally Verified	Zero bonus factor means min bonus equals PERCENTAGE_FACTOR	

PositionStatusMap Library

P-03. PositionStatusMap integrity

Status: Formally Verified

Rule Name	Status	Description	Link to rule report
setBorrowing	Formally Verified	Verifies that setBorrowing correctly updates the target reserve's flag and preserves all other reserve flags	
setUsingAsCollateral	Formally Verified	Verifies that setUsingAsCollateral correctly updates the target reserve's flag and preserves all other reserve flags	
isUsingAsCollateralOrBorrowing	Formally Verified	Verifies that the combined check correctly reflects the individual borrowing and collateral flags	
collateralCount	Formally Verified	Verifies that the collateralCount is correctly incremented or decremented when a reserve's collateral status changes.	*Run link*
next	Formally Verified	Verifies that the 'next' function correctly finds the nearest active reserve (borrowing or collateral) below the start index	
nextBorrowing	Formally Verified	Verifies that nextBorrowing correctly finds the nearest borrowing reserve below the start index	
nextCollateral	Formally Verified	Verifies that nextCollateral correctly finds the nearest collateral reserve below the start index	
neverReverts	Formally Verified	Ensures that all functions in PositionStatusMap never revert	

P-04. Function calculatePremiumRay summary

Status: Formally Verified

Rule Name	Status	Description	Link to rule report
calculatePremiumRay_equivalence	Formally Verified	Verifies that the Solidity implementation of calculatePremiumRay matches the symbolic CVL implementation.	*Run link*

SpokeUtils Library

P-05. Function toValue summary

Status: Formally Verified	

Rule Name	Status	Description	Link to rule report
checkToValueEquivalence	Formally Verified	Verify that SpokeUtils.toValue matches toValueCVL	*Run link*

ShareMath Library

P-06. Integrity of ShareMath functions

Status: Formally Verified

Rule Name	Status	Description	Link to rule report
toSharesUp_monotonicity	Formally Verified	Verifies that larger asset amounts result in larger or equal share amounts when rounding up.	*Run link*
toSharesUp_additivity	Formally Verified	Verifies that toSharesUp(x) + toSharesUp(y) >= toSharesUp(x+y), accounting for supply/asset changes	
toSharesUp_nonZero	Formally Verified	Ensures that any non-zero asset amount results in at least one share when rounding up	
toAssetsUp_monotonicity	Formally Verified	Verifies that larger share amounts result in larger or equal asset amounts when rounding up	
toAssetsUp_additivity	Formally Verified	Verifies that toAssetsUp(x) + toAssetsUp(y) >= toAssetsUp(x+y), accounting for supply/asset changes	
toSharesDown_monotonicity	Formally Verified	Verifies that larger asset amounts result in larger or equal share amounts when rounding down	
toSharesDown_additivity	Formally Verified	Verifies that toSharesDown(x) + toSharesDown(y) <= toSharesDown(x+y), accounting for supply/asset changes	
toAssetsDown_monotonicity	Formally Verified	Verifies that larger share amounts result in larger or equal asset amounts when rounding down	

toAssetsDown_additivity	Formally Verified	Verifies that toAssetsDown(x) + toAssetsDown(y) <= toAssetsDown(x+y), accounting for supply/asset changes	*Run link*

Math Libraries

P-07. Integrity of Math Functions

Status: Formally Verified

Rule Name	Status	Description	Link to rule report
MathUtils_mulDivDown	Formally Verified	Verifies that MathUtils.mulDivDown matches the symbolic mulDivDownCVL implementation	
MathUtils_mulDivUp	Formally Verified	Verifies that MathUtils.mulDivUp matches the symbolic mulDivUpCVL implementation	
WadRayMathExtended_rayMulDown	Formally Verified	Verifies that WadRayMath.rayMulDown matches the symbolic mulDivDownCVL(a, b, RAY) implementation.	
WadRayMathExtended_rayMulUp	Formally Verified	Verifies that WadRayMath.rayMulUp matches the symbolic mulDivUpCVL(a, b, RAY) implementation	
WadRayMathExtended_rayDivDown	Formally Verified	Verifies that WadRayMath.rayDivDown matches the symbolic mulDivDownCVL(a, RAY, b) implementation	Run link
WadRayMathExtended_rayDivUp	Formally Verified	Verifies that WadRayMath.rayDivUp matches the symbolic mulDivUpCVL(a, RAY, b) implementation	
WadRayMathExtended_wadDivDown	Formally Verified	Verifies that WadRayMath.wadDivDown matches the symbolic mulDivDownCVL(a, WAD, b) implementation	
WadRayMathExtended_wadDivUp	Formally Verified	Verifies that WadRayMath.wadDivUp matches the symbolic mulDivUpCVL(a,	

		WAD, b) implementatio
WadRayMathExtended_fromRayUp	Formally Verified	Verifies that WadRayMath.fromRayUp matches the symbolic divRayUpCVL implementation
WadRayMathExtended_toRay	Formally Verified	Verifies that WadRayMath.toRay matches the symbolic mulRayCVL implementation
percentMulDown_integrity	Formally Verified	Verifies that PercentageMath.percentMulDown matches the symbolic mulDivDownCVL implementation
percentMulDown_associativity	Formally Verified	Proves that the order of arguments (value vs percentage) does not change the result for percentMulDown.
percentMulUp_integrity	Formally Verified	Verifies that PercentageMath.percentMulUp matches the symbolic mulDivUpCVL implementation.
percentMulUp_associativity	Formally Verified	Proves that the order of arguments (value vs percentage) does not change the result for percentMulUp
WAD_definition	Formally Verified	Constant check for WAD (10^18)
RAY_definition	Formally Verified	Constant check for RAY (10^27)
PERCENTAGE_FACTOR_definition	Formally Verified	Constant check for PERCENTAGE_FACTOR (10000).

Disclaimer

Even though we hope this information is helpful, we provide no warranty of any kind, explicit or implied. The contents of this report should not be construed as a complete guarantee that the contract is secure in all dimensions. In no event shall Certora or any of its employees be liable for any claim, damages, or other liability, whether in an action of contract, tort, or otherwise, arising from, out of, or in connection with the results reported here.

About Certora

Certora is a Web3 security company that provides industry-leading formal verification tools and smart contract audits. Certora's flagship security product, Certora Prover, is a unique SaaS product that automatically locates even the most rare & hard-to-find bugs on your smart contracts or mathematically proves their absence. The Certora Prover plugs into your standard deployment pipeline and is widely used by developers and security researchers during audits and bug bounties.

Certora also provides architectural design reviews, where researchers analyze system design and trust boundaries early to identify structural risks and reduce complexity before implementation.

Certora conducts off-chain audits covering backend services, APIs, frontend, and mobile applications, focusing on vulnerabilities in authentication, data handling, key management, and wallet-related workflows.

Certora also offers Penetration Testing and Red Teaming to simulate real-world attacks and uncover exploitable weaknesses across infrastructure, applications, and business logic.

In addition, Certora provides operational security (OpSec) assessments, reviewing key management, access controls, and operational processes to strengthen overall security posture.

Finally, Certora delivers ongoing monitoring and incident response, enabling continuous threat detection and prevention, alert triage, and coordinated response to active incidents.